

PRISMA Visual Change Master Map - Ultra Technical Companion

- > Important: this file is a generated explanatory companion. It is not a competing authority.
- >
- > The canonical operator/lifecycle contract remains:
 - > prisma-html/docs/ops/PRISMA_VISUAL_CHANGE_MASTER_MAP.md
- >
- > Machine-readable authorities win inside their own domains. If this companion disagrees with current Factory Ledger, Authority Mesh, NDC, RIFAT,
- > Identity, Visual Source Manifest, Target Index, Visual Core, runtime evidence, or CI evidence, those current authorities win and this companion
- > must be regenerated/corrected.

0. What this companion adds

This ultra-technical view expands the canonical map so agents can understand not only the order of operations, but also the separation of responsibilities, readiness states, fail-closed gates, mutation contract, evidence contract, drift behavior and closure semantics.

It also includes a reviewed growth overlay that has become operationally relevant around:

- * Visual Promotion / Control Plane
- * Atlasfin-first normalization
- * Visual Current Truth
- * Surface Readiness
- * Work Entry Gate
- * scoped/sharded promotion
- * disjoint write ownership
- * baseHead freshness
- * zero-loss accounting

These orchestration concepts coordinate existing authorities. They do not replace NDC, RIFAT, Identity, UI Bridge, GVAE, Factory Ledger or Authority Mesh.

1. Three sentences that explain the system

- > NDC says what it means. Code Atlas says what exists. RIFAT says where it lands. Identity says how it should look. Mesh says what may be touched.
- > The generic engine in prisma-html applies the authorized visual change. Runtime QA proves what happened. Factory Ledger remembers the governed
- > truth.
- >
- > The neutral layer defines meaning. The specific layer defines location.
- >
- > Discovery is not binding. Binding is not authorization. Authorization is not mutation. Mutation is not certification.

These separations are hard constraints, not documentation style.

2. Canonical operating order

For a governed visual change, the working order is:

USER / OPERATOR
->
FACTORY LEDGER / ANTI-REWORK PROPOSAL
->
MASTER MAP / FIELD MANUAL
->
AUTHORITY MESH + LAYER MAP
->
REUSE CURRENT CENSUS
->
CODE ATLAS / UIMAP
->
NDC NEUTRAL MEANING
->
RIFAT EXACT VISUAL LOCATION
->
IDENTITY RECIPE / PROFILE
->
TARGET INDEX / BINDING RESOLUTION
->
VISUAL PROMOTION / CONTROL PLANE
->
WORK ENTRY GATE
->
CODE ATLAS UI BRIDGE PLAN
->
FACTORY LEDGER MUTATION GATE
->
GVAE PREVIEW
->
BEFORE EVIDENCE
->
GVAE APPLY
->
DETERMINISTIC PRODUCT PROJECTION
->
STATIC / SOURCE VALIDATION
->
RUNTIME VISUAL QA
->
DOMAIN VERIFIERS IF IMPACTED
->
CHANGE ASSURANCE OBSERVE / VERIFY / PROVE
->
PR / EXACT SHA CI
->
AUTOMESH REVALIDATION IF MAIN MOVES
->
MERGE / POST-MERGE
->
FACTORY LEDGER / EVIDENCE INDEX CLOSURE

Do not skip directly from "I found the CSS" to mutation.

3. Authority ownership

System / tool	Canonical job	May mutate product?
Factory Ledger	Capability maturity, anti-rework truth, evidence, nextGate, doNotRebuild	No
Universal anti-rework gate	Blocks duplicate/rebuild work	No
Authority Mesh	Task-exact authority, protected scope, Layer Map	No
AutoMesh v2	Revalidates/rebinds authority when canonical HEAD moves	No
Code Atlas core	Repository intelligence, impact, provenance, unknowns	No
UIMAP	Physical UI census / source map	No
NDC	Neutral meaning, scope, provenance concepts	No
RIFAT / prisma-ui	Exact visual location truth	Authority only
Identity Dictionary	Profiles, recipes, tokens, materials, states, adapters	Authority only
Atlasfin	Human visual cockpit, inspection, selection, preview, portable intent	No direct product write
Visual Source Manifest	Canonical-source -> runtime-projection contract	Descriptive authority
Generated Visual Target Index	Persistent physical application address book	Generated only
Binding Resolver	Meaning-to-target certification	No
Code Atlas UI Bridge	Deterministic read-only source plan	No
Visual Promotion / Control Plane	Candidate normalization, current truth, readiness, scoped promotion	No product mutation
Work Entry Gate	Fail-closed admission into governed visual mutation	No
Generic Visual Application Engine	Governed canonical visual mutation + projection + rollback/evidence	Yes, only after gates
Visual Core	Readiness computation and blockers	No
Mamastrophic / ScreensQA	Runtime/browser visual evidence	No source mutation
Domain verifiers	Specialized runtime/data/licensing/sync verification when impacted	No visual authority
Change Assurance	UNDERSTAND -> PROVE evidence lifecycle	No authority invention
GitHub Actions	Exact-SHA execution rail	Workflow-controlled
AutoGit	Git/PR mechanics only	Git logistics
Factory Ledger closure	Records final governed truth	No product mutation

4. Neutral meaning versus implementation

Wrong canonical model:

TB_POS_SALE
PC_SALE_PANEL
MB_SALE_WIDGET

Correct model:

ENT.sale
EVT.sale.created
ACT.sale.checkout
MET.sales.today

Then project the neutral concept into specific surfaces:

neutral visual meaning
->

identity recipe / profile
->
surface adapter
->
specific route / region / slot / component / layer
->
runtime projection

A UI implementation locates meaning. It does not define the canonical meaning.

5. Census, target address, meaning, binding and application are different truths

CENSUS / UIMAP
"this physically exists"
->
TARGET INDEX
"this is its persistent proven address"
->
NDC / VISUAL MEANING
"this is what it means"
->
BINDING
"this meaning is certified to this exact target"
->
APPLICATION
"an authorized recipe may be applied here"
->
RUNTIME CERTIFICATION
"the intended result actually rendered"

Therefore:

- * a surface can be highly censused while many targets remain SEMANTIC_PENDING;
- * a target can be SOURCE_RESOLVED but not application-ready;
- * bindingCount must never be interpreted as total UI census count;
- * runtime certification remains separate from source/static readiness.

6. Generated Visual Target Index contract

The Target Index exists so PRISMA does not repeat broad discovery and does not confuse physical inventory with semantic readiness.

Canonical generated family:

prisma-html/
authority/
rifat/
prisma-ui/

visual-control/
target-index/
manifest.json
tablet.jsonl
pc.jsonl
mobile.jsonl
web.jsonl
chart-lab.jsonl
control-center.jsonl
shared-ui.jsonl

A minimum target record should preserve at least:

surfaceld
interfaceld
routeId
routePath
regionId
slotId
componentId
componentUild
ownerId
ownerFile
ownerSymbol
renderSourceFile
renderSymbol
visualTargets[]
styleSourceFile
selector
visualTargetId
implementationLayerId
neutralMeaningId
bindingId
layerId
adapterId
sourceHashes{}
evidenceRefs[]
targetResolutionStatus
semanticStatus
applicationReadiness
blockingReasons[]

Important status example:

targetResolutionStatus = SOURCE_RESOLVED
semanticStatus = SEMANTIC_PENDING
applicationReadiness = BLOCKED

This means the physical target is proven, but mutation is not authorized.

7. Binding envelope

A governed binding is not a fuzzy name match.

Conceptually:

neutral visual meaning
+
recipe / profile
+
surface
+
exact target coordinates
+
certified layer
=
binding envelope

A binding becomes RESOLVED only from direct evidence.

If the surface census is current but the binding is missing, do not recensus the surface. Promote or author only the missing semantic/binding contract.

8. Readiness state machine

Every visual object should be representable through this readiness chain:

DISCOVERED
->
PHYSICAL_RESOLVED
->
SEMANTIC_RESOLVED
->
VISUAL_CONTROL_RESOLVED
->
BINDING_CERTIFIED
->
RECIPE_COMPATIBLE
->
PROJECTION_RESOLVED
->
AUTHORITY_PREFLIGHT_ELIGIBLE
->
APPLY_ELIGIBLE
->
SOURCE_APPLIED
->
RUNTIME_VISUAL_PENDING
->
VISUAL_CERTIFIED

Valid fail-closed states include:

UNMAPPED
PARTIAL
BLOCKED_BY_CONFLICT
BLOCKED_BY_MISSING_OWNER
BLOCKED_BY_MISSING_SOURCE
BLOCKED_BY_MISSING_LAYER
BLOCKED_BY_MISSING_BINDING
BLOCKED_BY_AMBIGUOUS_BINDING
BLOCKED_BY_SCOPE
BLOCKED_BY_DRIFT
BLOCKED_BY_STALE_AUTHORITY
BLOCKED_BY_NO_CANONICAL_VISUAL_SOURCE
BLOCKED_BY_RECIPE_INCOMPATIBILITY
BLOCKED_BY_RUNTIME_EVIDENCE
DRIFTED

UNKNOWN, PARTIAL and BLOCKED are legitimate outcomes. They are not failed reporting.

9. Surface Readiness

The system should generate readiness from current authority/evidence rather than hand-maintained percentages.

Recommended generated columns:

Surface
Census current
Route resolved
Owner resolved
Region resolved
Slot resolved
Visual target resolved
Physical layer resolved
NDC meaning
Binding
Recipe
Projection
Authority eligible
Generic apply
Source applied
Runtime visual
Final status

A percentage is not authority. Generated evidence-backed state is authority input.

10. Visual Promotion / Control Plane growth overlay

The reviewed orchestration layer is:

Candidate intake
->
Atlasfin-first normalization
->
Visual Current Truth
->
Surface Readiness
->
Work Entry Gate
->
scoped promotion / governed downstream plan

It checks, where applicable:

- * authority-qualified IDs;
- * vocabulary validity;
- * duplicate/collision conditions;
- * current baseHead;
- * source hash freshness;
- * disjoint lane/write ownership;
- * current truth;
- * readiness blockers;
- * zero-loss accounting.

It can produce:

- * promotion-ready request;
- * explicit blockers;
- * readiness evidence;
- * scope/write-owner decision;
- * candidate collision evidence.

It may not:

- * invent NDC meaning;
- * grant itself mutation authority;
- * patch runtime directly;
- * override Factory Ledger or Mesh;
- * convert UNKNOWN or BLOCKED to green.

11. Universal Work Entry Gate

Before visual source mutation, the repository-level entry decision must be fail-closed.

Current root AGENTS.md defines the canonical entry gate:

PYTHONPATH=prisma-html/tools python -m visual_application.visual_work_entry_gate --request <request.json>

The recognized paths are:

- * GVAE_EXACT_APPLY: continue through normal GVAE PREVIEW/APPLY/receipt/VERIFY only with valid mutation authority;

* SURFACE_BATCH_PLAN: read-only bounded planning, never wildcard APPLY;
* REGISTER_TARGET_FIRST: establish missing meaning, recipe, binding, layer policy, adapter/projection authority and exact target identity first;
* BLOCKED: stop.

Do not edit around the gate.

12. Code Atlas UI Bridge contract

Inputs:

resolved target
compatible recipe
current source
adapter
binding
layer

Outputs:

PLAN
semantic diff
operations
blocking reasons
checksum / digest

No source mutation occurs here.

Code Atlas discovers, resolves, observes and plans. It does not become visual authority.

13. GVAE canonical home

The Generic Visual Application Engine belongs under prisma-html, because prisma-html is the editable visual authority.

Expected implementation family:

prisma-html/
tools/
visual_application/
engine.py
preflight.py
request.py
resolver.py
projection.py
rollback.py
evidence.py
integrity.py

safety.py
writers/
css.py
css_module.py
json_tokens.py
generated_projection.py
validators/
request_validator.py
target_validator.py
plan_validator.py
projection_validator.py
evidence_validator.py

Initial bounded writer scope:

- * CSS;
- * CSS Modules;
- * governed JSON tokens/recipes;
- * declared generated projections.

Arbitrary TSX/DOM structure and business handlers are not visual-only mutation. Structural React changes require a separately governed implementation path.

14. GVAE modes

Required modes:

preview
apply
verify
rollback

Rules:

- * preview never mutates;
- * apply requires current gated authorization;
- * verify verifies exact post-application state and evidence;
- * rollback restores exact BEFORE bytes and reruns relevant validation.

Write policy:

prisma-html canonical visual source
->
GVAE
->
deterministic projection generator
->
product runtime output

Do not manually edit generated product projections.

If no canonical visual source is declared:

BLOCKED_BY_NO_CANONICAL_VISUAL_SOURCE

The next task is source promotion/governance, not an ad-hoc runtime patch.

15. GVAE preflight

Before any write, require the applicable set of:

- Factory Ledger PROPOSAL PASS
- fresh/revalidated exact-task Authority Mesh
- Factory Ledger MUTATION PASS
- Layer Map present
- exact target SOURCE_RESOLVED
- neutral meaning certified where required
- binding RESOLVED
- canonical layer certified
- surface adapter compatible
- recipe compatible
- UI Bridge plan reviewed
- plan digest valid
- source hashes current
- canonical visual source declared
- projection output declared
- surface scope allowed
- excluded surfaces untouched
- backup destination available
- BEFORE evidence present where required

One failed mandatory precondition means no mutation.

16. GVAE transaction and rollback

On an exact reviewed plan:

- recheck all preconditions
- >
- write exact backups
- >
- patch canonical visual authority atomically
- >
- run declared deterministic projection
- >
- verify runtime/product output
- >

verify hashes
->
record transaction
->
collect reproducible evidence

If any write or verification step fails:

automatic exact-byte rollback
->
revalidation
->
explicit failed/blocked result

Required invariants include:

- * no !important;
- * no surface expansion;
- * no unknown target guessing;
- * no direct generated-output editing;
- * no mutation on stale source hash;
- * no mutation without rollback;
- * no static-only certification when runtime visual proof is required;
- * no functional-smoke-only visual certification;
- * no hidden skipped state;
- * no product-wide READY from one target certification.

17. Golden Cobrar compatibility path

The existing Cobrar transaction remains a historical/certified reference.

The generic engine must prove parity against the certified Cobrar case before a Cobrar-specific implementation is reduced to a compatibility wrapper.

Do not delete certified historical evidence merely because a generalized engine exists.

18. Full visual procedure, steps 1-14

1. Normalize the user request: surface, object, intent, constraints and excluded scope.
2. Resolve current canonical HEAD.
3. Read Factory Ledger and run anti-rework PROPOSAL.
4. Read the canonical Master Map and Field Manual.
5. Generate/revalidate task-exact Authority Mesh and Layer Map.
6. Reuse current census before rediscovery.
7. Resolve the exact physical target with Code Atlas/UIMAP.
8. Resolve neutral meaning with NDC.
9. Resolve exact visual location with RIFAT/prisma-ui.
10. Resolve visual intent with Identity Dictionary.

- 11. Resolve/promote the exact element binding.
- 12. Pass candidate/current-truth/readiness checks through the Control Plane and Work Entry Gate where applicable.
- 13. Build deterministic Code Atlas UI Bridge plan and review exact impact.
- 14. Run Factory Ledger MUTATION gate.

If exact target selection remains ambiguous at any point, stop.

19. Full visual procedure, steps 15-26

- 15. Capture BEFORE evidence where runtime visual proof is required.
- 16. Run GVAE preview.
- 17. On exact reviewed plan, run GVAE apply with backups and automatic rollback semantics.
- 18. Run narrow static/source validation.
- 19. Run runtime visual QA on the same route, viewport, browser/device, target and state model.
- 20. Run domain-specific verifiers only when Authority Mesh/impact evidence says they apply.
- 21. Run Change Assurance OBSERVE / VERIFY / PROVE.
- 22. Run PR and exact-SHA CI.
- 23. If main moves, revalidate with AutoMesh v2.
- 24. Pass merge guard and merge only the reviewed certified exact scope.
- 25. Verify actual canonical main.
- 26. Update Factory Ledger / Evidence Index only if capability truth actually changed.

20. Runtime visual QA contract

AFTER evidence must be comparable to BEFORE evidence.

Where applicable, require:

- same route
- same viewport
- same browser/device
- same target
- same relevant state model
- expected visual change present
- geometry acceptable
- no overflow/collision
- required states checked
- no new console errors
- no target network failures
- reduced-motion behavior
- accessibility invariants

Skipped states must remain explicit.

Static PASS is not visual PASS. Functional PASS is not visual PASS.

21. Drift and AutoMesh behavior

When canonical main moves, do not blindly retain stale authority and do not blindly destroy otherwise-valid authority.

Expected revalidation outcomes include:

PASS_ALREADY_CURRENT
PASS_NO_RELEVANT_DRIFT
BLOCKED_RELEVANT_DRIFT
BLOCKED_NON_ANCESTOR_DRIFT
BLOCKED_INVALID_PRIOR_AUTHORITY

Only valid rebound/fresh authority may continue.

No downgrade rule:

- * if runtime contains legitimate newer governed work, reconcile/promote it;
- * otherwise restore the deterministic projection from current canonical authority;
- * never overwrite legitimate newer governed runtime merely to make a validator green.

22. Golden examples

A. PC target already censused but binding pending

User asks for visual change
->
Ledger proposal
->
Mesh
->
reuse current PC UIMAP / Visual Control target
->
DO NOT broad-recensus PC
->
resolve NDC meaning
->
author/certify missing identity-layer + binding
->
UI Bridge plan
->
mutation gate
->
GVAE
->
PC projection
->
runtime visual QA
->

Change Assurance
->
PR/merge
->
Ledger

B. Target physically missing

User asks for target
->
Ledger + Mesh
->
UIMAP cannot prove target
->
UNMAPPED / BLOCKED
->
target-specific discovery or source task

Do not invent a component ID, binding or layer.

C. Runtime differs from RIFAT

RIFAT source hash != product output hash
->
inspect exact-copy drift and Git/current authority
->
if runtime contains legitimate newer governed work:
reconcile/promote without downgrade
else:
restore deterministic projection from canonical authority
->
refresh hashes mechanically
->
revalidate

23. Permanent anti-confusion rules

1. A census is not a binding registry.
2. A binding registry is not the full UI inventory.
3. A physical target can be known while semantic binding remains pending.
4. NDC meaning must not be inferred from an app-specific filename.
5. RIFAT location must not become neutral meaning.
6. Code Atlas plans and observes; it does not become visual authority.
7. Atlasfin operates visual authority but does not write product directly.
8. GVAE belongs in prisma-html.
9. Generated product outputs are projections, not competing editable authorities.
10. Factory Ledger and Mesh bracket the work: proposal before planning, mutation gate before writing.

- 11. Broad rediscovery is forbidden when current exact evidence already proves the target.
- 12. UNKNOWN, PARTIAL and BLOCKED are valid outcomes.
- 13. Functional PASS is not visual PASS.
- 14. Static/source-ready is not runtime visual certification.
- 15. One certified target does not certify a whole surface.
- 16. No final visual claim without reproducible runtime evidence where the contract requires it.
- 17. No visual task starts by editing CSS. It starts by resolving governed truth.

24. Agent startup rule

A context-free agent performing PRISMA visual work must start from the repository, not conversational memory.

At minimum:

- 1. read AGENTS.md;
- 2. read apps/terminal-de-venta-system/docs/ops/PRISMA_FIELD_MANUAL_APRENDIZAJE_OPERATIVO.md;
- 3. read PRISMA Factory Ledger/PRISMA_FACTORY_LEDGER_AGENT_GATE.md;
- 4. read prisma-html/docs/ops/PRISMA_VISUAL_CHANGE_MASTER_MAP.md;
- 5. obtain or revalidate task-exact Authority Mesh + Layer Map;
- 6. use the canonical Work Entry Gate before visual mutation;
- 7. do not bypass the resulting decision.

The repository gate and CI remain authoritative when conversational context is missing.

25. Final doctrine

PRISMA visual work must never collapse to:

find CSS
change CSS
refresh browser
hope

The governed model is:

know existing capability
->
authorize the task
->
reuse the census
->
identify neutral meaning
->
resolve exact visual coordinates
->
select governed visual intent
->
certify the binding

->
establish current truth/readiness
->
pass work entry
->
plan deterministically
->
authorize mutation
->
apply through the canonical visual engine
->
project deterministically
->
prove runtime outcome
->
close evidence and maturity

That is how a visual edit becomes a traceable, scoped, reversible and certifiable PRISMA operation.